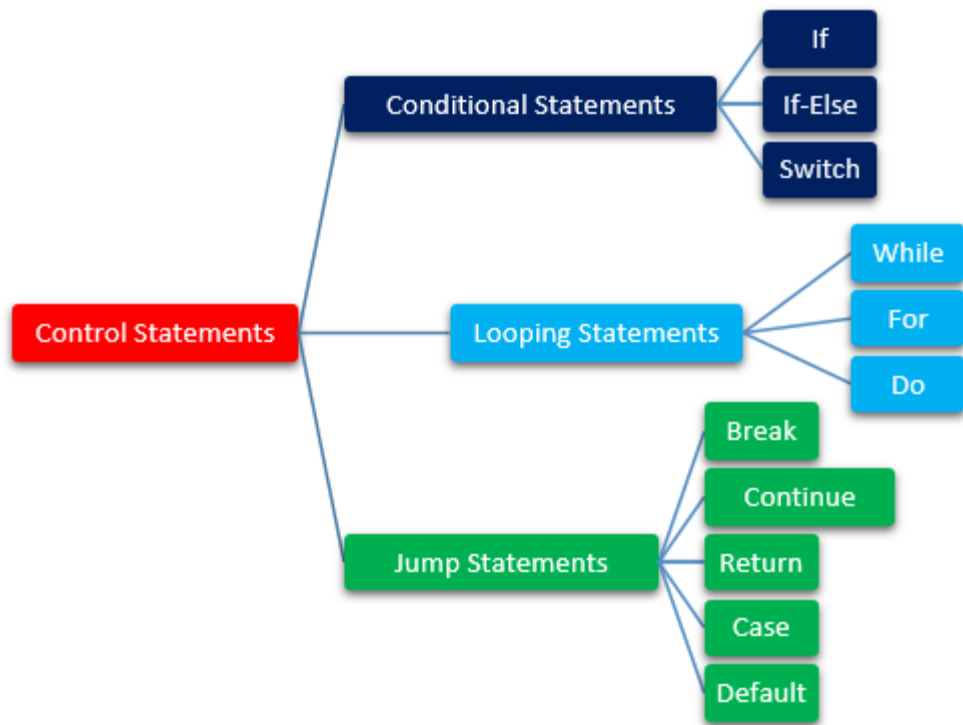


Java

Conditional Statements in Java :-

- Conditional statements control the flow of program execution.
- They allow decisions to be made based on conditions (true/false).
- Different code blocks execute depending on the condition.



Types of Conditional Statements :-

1) switch Statement

- Used to execute one block of code from multiple options based on the value of an expression.
- It is an alternative to if-else-if ladder when many conditions are checked.

Syntax:

```
switch (expression) {  
    case value1:  
        // code to be executed if expression == value1  
        break;  
  
    case value2:  
        // code to be executed if expression == value2  
        break;  
  
    case value3:
```

```
    // code to be executed if expression == value3
break;

default:
    // code to be executed if none of the cases match
}
```

Example –

```
package Basic_Concept;

public class switchstatement {

    public static void main(String[] args) {
        int num=20;
        switch(num)
        {
            case 10:
                System.out.println("Number is 10");
                break;

            case 20:
                System.out.println("Number is 20");
                break;

            case 30:
                System.out.println("Number is 30");
                break;

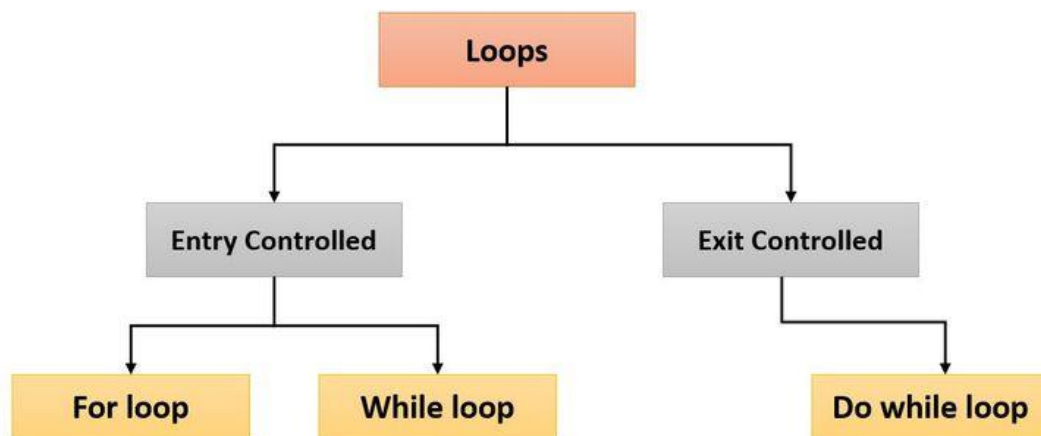
            default:
                System.out.println("NONE");
        }
    }
}
```

Output –

Number is 20

Looping Statements in Java –

- Looping statements are used to execute a block of code repeatedly.
- They help to reduce code repetition and improve efficiency.
- Every loop has initialization, condition, and update.
- Java provides four types of loops:



1) For Loop –

- Used when the number of iterations is known.

Syntax:

```
for (initialization; condition; update) {  
  
    // code to be executed  
}
```

Example –

```
package Basic_Concept;  
  
public class forloopdemo {  
  
    public static void main(String[] args) {  
        //print your name 4 times  
  
        for(int i=1;i<=4;i++)  
        {  
            System.out.println("Sagar");  
        }  
    }  
}
```

}

Output –

Sagar

Sagar

Sagar

Sagar

2) While Loop –

- Used when the number of iterations is not known in advance.
- Condition is checked before executing the loop.

Syntax:

```
while (condition) {  
    // code to be executed  
}
```

Example –

```
package Basic_Concept;  
  
public class whiledemo {  
    public static void main(String[] args) {  
  
        //print the numbers from 1 to 10  
  
        int n=1;  
  
        while(n<=10)  
        {  
            System.out.println(n);  
            n++;  
        }  
    }  
}
```

Output –

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

3) Do - While Loop –

- Similar to while loop, but the condition is checked after executing the loop body.
- Guarantees execution at least once.

Syntax:

```
do {  
    // code to be executed  
} while (condition);
```

Example –

```
package Basic_Concept;  
  
public class dowhiledemo {  
    public static void main(String[] args) {  
  
        //print 40-52  
  
        int num=40;  
  
        do {  
            System.out.println(num);  
            num++;  
        }while(num<=52);  
    }  
}
```

Output –

```
40  
41  
42  
43  
44  
45  
46  
47  
48  
49  
50  
51  
52
```

Difference Between while loop & do-while loop :-

Aspect	while Loop	Do while Loop
Execution Condition	The condition is checked before execution of the loop body.	The condition is checked after the loop body has executed.
Minimum Iterations	The loop may not execute at all if the condition is false initially.	The loop will always execute at least once, regardless of the condition.
Syntax	<code>while (condition) { // code }</code>	<code>do { // code } while (condition);</code>
Use Case	Ideal when the loop should only run if the condition is initially true.	Ideal when the loop must run at least once, even if the condition is false.
Example	<code>while (x < 10) { x++; }</code>	<code>do { x++; } while (x < 10);</code>